

Herrera_Christian_Lab10

April 3, 2026

1 Week 12. Lab 11- Advance Pandas (Questions)

2 Assignment Submission Guidelines

2.0.1 Steps to Submit Your Lab Assignments Using Google Colab

1. Submission Deadline:

- All assignments must be submitted **no later than 23:59 PM tonight**.
- Late submissions will not be accepted unless prior arrangements have been made by the TAs.

2. Complete Your Work:

- Finish your work in Google Colab and ensure that your notebook is saved.

3. Submit Your Assignment on Canvas:

- Log in to **Canvas** and navigate to the assignment submission page.

4. File Naming Convention:

- Please name your files as follows: Lastname_Firstname_AssignmentName
- Example: Alex_John_Lab10.ipynb, Alex_John_Lab10.pdf

5. Technical Issues:

- If you encounter any technical issues with Canvas or your submission, please contact the TAs immediately **before the deadline** to avoid penalties.

3 Questions

3.1 Question 1

Load the Movies dataset into a pandas DataFrame and print the head of the dataset.

```
[168]: import pandas as pd
import matplotlib.pyplot as plt

# reading the csv.
df = pd.read_csv('Movies.csv')

# The head.
df.head()
```

```
[168]:
```

	Film	Genre	RottenTomatoes_Ratings	Audience Ratings	\
0	(500) Days of Summer	Comedy	87.0	81.0	
1	10,000 B.C.	Adventure	9.0	44.0	
2	12 Rounds	Action	30.0	52.0	
3	127 Hours	Adventure	93.0	84.0	
4	17 Again	Comedy	55.0	70.0	

	Budget_m	Year_release
0	8.0	2009.0
1	105.0	2008.0
2	20.0	2009.0
3	18.0	2010.0
4	20.0	2009.0

3.2 Question 2

Find the number of rows and columns in the dataset.

```
[169]: print(f'rows: {df.shape[0]}, columns: {df.shape[1]}')
```

```
rows: 559, columns: 6
```

3.3 Question 3

Check for missing values in each column.

```
[170]: # Sum of nulls.
df.isnull().sum()
```

```
[170]: Film          0
Genre          2
RottenTomatoes_Ratings  8
Audience Ratings 11
Budget_m       9
Year_release   3
dtype: int64
```

3.4 Question 4

Display basic statistics for the ‘Rotten Tomatoes Ratings’ and ‘Budget’ columns.

```
[171]: # Summary statistics.  
df[['RottenTomatoes_Ratings', 'Budget_m']].describe()
```

```
[171]:
```

	RottenTomatoes_Ratings	Budget_m
count	551.000000	550.000000
mean	47.477314	50.390909
std	26.268386	48.986726
min	0.000000	0.000000
25%	25.500000	20.000000
50%	46.000000	35.000000
75%	70.000000	65.000000
max	97.000000	300.000000

3.5 Question 5

Create a new DataFrame containing only the films released after the year 2000.

```
[172]: # After 2000.  
after_2000_df = df[df['Year_release'] > 2000].copy()  
  
after_2000_df
```

```
[172]:
```

	Film	Genre	RottenTomatoes_Ratings	\
0	(500) Days of Summer	Comedy	87.0	
1	10,000 B.C.	Adventure	9.0	
2	12 Rounds	Action	30.0	
3	127 Hours	Adventure	93.0	
4	17 Again	Comedy	55.0	
..	...	...	...	
554	Your Highness	Comedy	26.0	
555	Youth in Revolt	Comedy	68.0	
556	Zodiac	Thriller	89.0	
557	Zombieland	Action	90.0	
558	Zookeeper	Comedy	14.0	

Audience Ratings Budget_m Year_release

0	81.0	8.0	2009.0
1	44.0	105.0	2008.0
2	52.0	20.0	2009.0
3	84.0	18.0	2010.0
4	70.0	20.0	2009.0
..	...	...	...
554	36.0	50.0	2011.0
555	52.0	18.0	2009.0
556	73.0	65.0	2007.0
557	87.0	24.0	2009.0
558	42.0	NaN	2011.0

[556 rows x 6 columns]

3.6 Question 6

Filter the dataset to include only films with a budget greater than \$50 million

```
[173]: df[df['Budget_m'] > 50]
```

```
[173]:
```

	Film	Genre	RottenTomatoes_Ratings \
1	10,000 B.C.	Adventure	9.0
5	2012	Action	39.0
19	Alice in Wonderland	Adventure	52.0
25	American Gangster	Thriller	79.0
32	Australia	Adventure	55.0
..	...	...	...
548	Year One	Adventure	14.0
549	Yes Man	NaN	43.0
550	Yogi Bear	Comedy	14.0
552	You Don't Mess with the Zohan	Comedy	36.0
556	Zodiac	Thriller	89.0

	Audience Ratings	Budget_m	Year_release
1	44.0	105.0	2008.0
5	63.0	200.0	2009.0
19	72.0	200.0	2010.0
25	87.0	100.0	2007.0
32	69.0	78.0	2008.0
..	...	...	...
548	31.0	60.0	2009.0
549	72.0	70.0	NaN
550	36.0	80.0	2010.0

552	50.0	80.0	2008.0
556	73.0	65.0	2007.0

[176 rows x 6 columns]

3.7 Question 7

Sort the dataset by 'Rotten Tomatoes Ratings %' in descending order.

```
[174]: rotten_tomato_df = df.sort_values('RottenTomatoes_Ratings', ascending=False)

# Top 10 according to rotten tomatoes.
rotten_tomato_df.head(10)
```

```
[174]:
```

	Film	Genre	RottenTomatoes_Ratings \
431	The Hurt Locker	Drama	97.0
387	The Artist	Drama	97.0
455	The Muppets	Comedy	97.0
506	True Grit	Adventure	96.0
275	No Country for Old Men	Drama	95.0
249	Moneyball	Drama	95.0
440	The Kids Are All Right	Comedy	94.0
207	Juno	Comedy	94.0
196	Iron Man	Action	94.0
545	Winter's Bone	Drama	94.0

	Audience Ratings	Budget_m	Year_release
431	83.0	15.0	2008.0
387	91.0	16.0	2011.0
455	87.0	45.0	2011.0
506	86.0	38.0	2010.0
275	84.0	25.0	2007.0
249	89.0	50.0	2011.0
440	72.0	4.0	2010.0
207	89.0	8.0	2007.0
196	91.0	186.0	2008.0
545	73.0	2.0	2010.0

3.8 Question 8

Sort the dataset by 'Year of release' in ascending order.

```
[175]: year_release_sort_df = df.sort_values('Year_release', ascending=True)

# Top 10 oldest first.
year_release_sort_df.head(10)
```

```
[175]:
```

	Film	Genre	RottenTomatoes_Ratings \
556	Zodiac	Thriller	89.0
20	Alien vs. Predator -- Requiem	Horror	14.0
25	American Gangster	Thriller	79.0
517	Vacancy	Comedy	55.0
525	Waitress	Romance	89.0
526	Walk Hard: The Dewey Cox Story	Comedy	75.0
511	Under the Same Moon	Drama	72.0
510	Tyler Perry's Why Did I get Married	Romance	46.0
45	Beowulf	Action	71.0
48	Blades of Glory	Comedy	69.0

	Audience Ratings	Budget_m	Year_release
556	73.0	65.0	2007.0
20	37.0	40.0	2007.0
25	87.0	100.0	2007.0
517	46.0	19.0	2007.0
525	67.0	2.0	2007.0
526	NaN	35.0	2007.0
511	37.0	2.0	2007.0
510	47.0	15.0	2007.0
45	52.0	150.0	2007.0
48	66.0	61.0	2007.0

3.9 Question 9

Group the data by 'Genre' and calculate the mean 'Rotten Tomatoes Rating %' for each genre. Also, sort the Genres from highest to lowest Rotten Tomatoes Ratings

```
[176]: df.groupby(by='Genre')['RottenTomatoes_Ratings'].mean().
      ↪sort_values(ascending=False)
```

```
[176]: Genre
Thriller    58.085714
Drama       56.180000
```

```

Adventure    53.103448
Comedy       45.734940
Romance      45.388889
Action       44.588235
Horror       35.020833
Name: RottenTomatoes_Ratings, dtype: float64

```

3.10 Question 10

Calculate the average budget for films released in each year

```
[177]: avg_budget_arr = df.groupby(by='Year_release')['Budget_m'].mean()
avg_budget_arr
```

```

[177]: Year_release
2007.0    58.714286
2008.0    46.905983
2009.0    48.587719
2010.0    50.598291
2011.0    50.221311
Name: Budget_m, dtype: float64

```

3.11 Question 11

Create a bar chart to show the average 'Rotten Tomatoes Ratings %' for each genre.

```

[178]: new_df = df.groupby('Genre')['RottenTomatoes_Ratings'].mean()

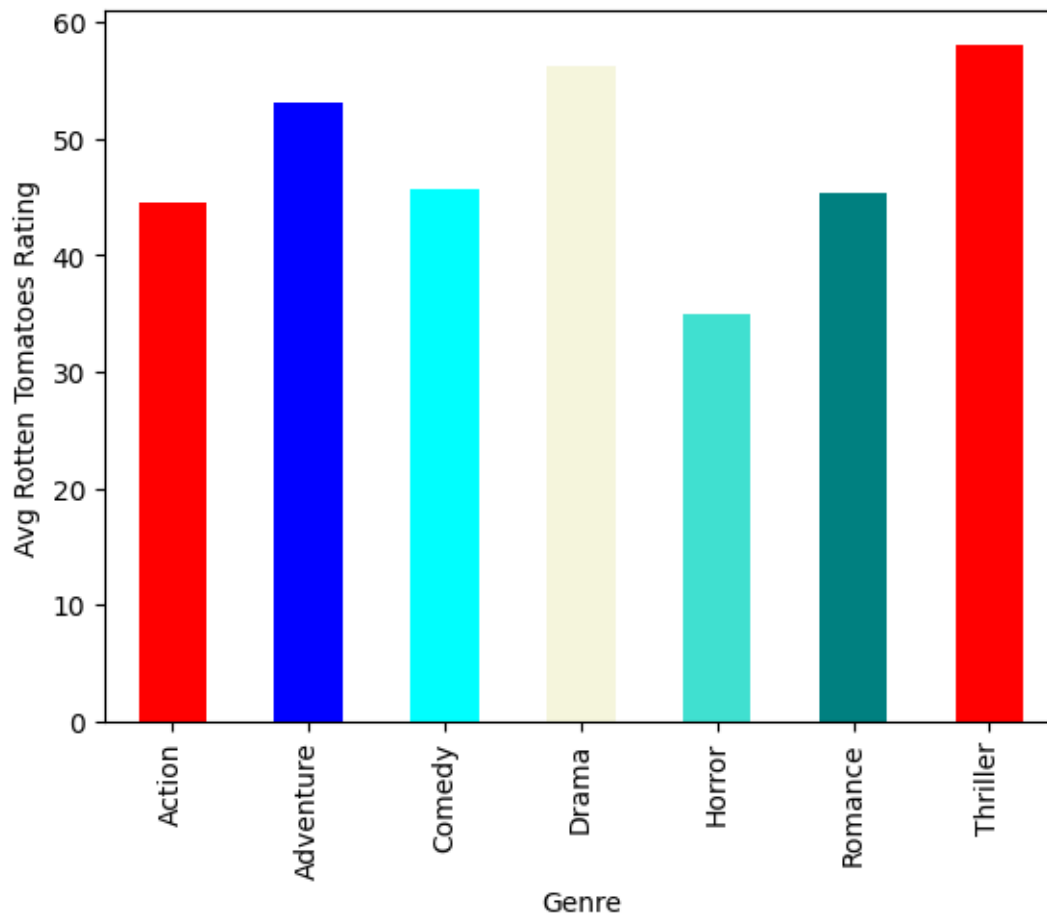
color_list = ['red', 'blue', 'cyan', 'beige', 'turquoise', 'teal'] # The_
    ugliest colors I could put together.

new_df.plot(kind='bar', color=color_list)

plt.xlabel('Genre')
plt.ylabel('Avg Rotten Tomatoes Rating')

```

```
[178]: Text(0, 0.5, 'Avg Rotten Tomatoes Rating')
```



3.12 Question 12

Replace any missing values in the 'Budget (million \$)' column with the mean budget of the dataset

```
[179]: # The mean
mean_budget = df['Budget_m'].mean()

# Filling with the mean.
df['Budget_m'] = df['Budget_m'].fillna(mean_budget)

df.head()
```



```
[179]:
```

	Film	Genre	RottenTomatoes_Ratings	Audience Ratings	\
0	(500) Days of Summer	Comedy	87.0	81.0	
1	10,000 B.C.	Adventure	9.0	44.0	
2	12 Rounds	Action	30.0	52.0	
3	127 Hours	Adventure	93.0	84.0	
4	17 Again	Comedy	55.0	70.0	

	Budget_m	Year_release
0	8.0	2009.0
1	105.0	2008.0
2	20.0	2009.0
3	18.0	2010.0
4	20.0	2009.0

3.13 Question 13

Convert the 'Year of release' column to a datetime format.

```
[180]: df['Year_release'] = pd.to_datetime(df['Year_release'], format='%Y')
df.head()
```

```
[180]:
```

	Film	Genre	RottenTomatoes_Ratings	Audience Ratings	\
0	(500) Days of Summer	Comedy	87.0	81.0	
1	10,000 B.C.	Adventure	9.0	44.0	
2	12 Rounds	Action	30.0	52.0	
3	127 Hours	Adventure	93.0	84.0	
4	17 Again	Comedy	55.0	70.0	

	Budget_m	Year_release
0	8.0	2009-01-01
1	105.0	2008-01-01
2	20.0	2009-01-01
3	18.0	2010-01-01
4	20.0	2009-01-01

3.14 Question 14

Write code to perform mean imputation for missing values in the ‘Audience Ratings’ column

```
[181]: # Same process as the other question.
mean_audience = df['Audience Ratings'].mean()

df['Audience Ratings'] = df['Audience Ratings'].fillna(mean_audience)

df.head()
```

```
[181]:
```

	Film	Genre	RottenTomatoes_Ratings	Audience Ratings \
0	(500) Days of Summer	Comedy	87.0	81.0
1	10,000 B.C.	Adventure	9.0	44.0
2	12 Rounds	Action	30.0	52.0
3	127 Hours	Adventure	93.0	84.0
4	17 Again	Comedy	55.0	70.0

	Budget_m	Year_release
0	8.0	2009-01-01
1	105.0	2008-01-01
2	20.0	2009-01-01
3	18.0	2010-01-01
4	20.0	2009-01-01

3.15 Question 15

Create a DataFrame that contains only movies in the ‘Action’ genre that have a ‘RottenTomatoes_Ratings’ greater than 80. How many movies are there in this filtered DataFrame?

```
[182]: good_action_df = df[(df['Genre'] == 'Action') & (df['RottenTomatoes_Ratings'] > 80)]

display(good_action_df)

print(f'There are {good_action_df.shape[0]} good action movies.')
```

	Film	Genre	RottenTomatoes_Ratings \
33	Avatar	Action	83.0
94	District 9	Action	91.0
155	Grindhouse	Action	82.0
170	Hellboy II: The Golden Army	Action	87.0
190	Inception	Action	86.0

196	Iron Man	Action	94.0
248	Mission Impossible 4	Action	93.0
326	Rise of the Planet of the Apes	Action	83.0
334	Scott Pilgrim vs. the World	Action	81.0
362	Star Trek	Action	94.0
514	Unstoppable	Action	86.0
530	Warrior	Action	83.0
547	X-Men: First Class	Action	87.0
557	Zombieland	Action	90.0

	Audience Ratings	Budget_m	Year_release
33	92.0	237.0	2009-01-01
94	81.0	30.0	2009-01-01
155	86.0	53.0	2007-01-01
170	70.0	83.0	2008-01-01
190	93.0	160.0	2010-01-01
196	91.0	186.0	2008-01-01
248	86.0	145.0	2011-01-01
326	87.0	93.0	2011-01-01
334	83.0	60.0	2010-01-01
362	91.0	140.0	2009-01-01
514	73.0	95.0	2010-01-01
530	93.0	25.0	2011-01-01
547	88.0	160.0	2011-01-01
557	87.0	24.0	2009-01-01

There are 14 good action movies.

3.16 Question 16

Group the movies by ‘Year_release’ and find the average ‘Budget_m’ for each year. Return a DataFrame of the years and their corresponding average budgets.

```
[183]: year_budget_group = df.groupby('Year_release')['Budget_m'].mean()

year_budget_group
```

```
[183]: Year_release
2007-01-01    58.607576
2008-01-01    46.993106
2009-01-01    48.618809
2010-01-01    50.594805
2011-01-01    50.222690
Name: Budget_m, dtype: float64
```

3.17 Question 17

Create a new column 'Profitable' that contains boolean values. A movie is considered profitable if the 'Audience Ratings' is above 60 and the 'Budget_m' is below the median budget of all movies in the dataset. Show the head of the DataFrame with the new column.

```
[184]: median_budget = df['Budget_m'].median()

# Making a new col based on the conditions.
df['Profitable'] = (df['Audience Ratings'] > 60) & (df['Budget_m'] < median_budget)

df.head()
```

```
[184]:
```

	Film	Genre	RottenTomatoes_Ratings	Audience Ratings	\
0	(500) Days of Summer	Comedy	87.0	81.0	
1	10,000 B.C.	Adventure	9.0	44.0	
2	12 Rounds	Action	30.0	52.0	
3	127 Hours	Adventure	93.0	84.0	
4	17 Again	Comedy	55.0	70.0	

	Budget_m	Year_release	Profitable
0	8.0	2009-01-01	True
1	105.0	2008-01-01	False
2	20.0	2009-01-01	False
3	18.0	2010-01-01	True
4	20.0	2009-01-01	True